STAT 405 Final Project:
Bayesian Inference for NBA Player Salary Valuation

Yumeng Wu(544207773), Desheng Lin(29605730)
GitHub Repo: https://github.com/heydevin/stat405-final-project

1 Introduction

The National Basketball Association (NBA) is one of the most commercially significant professional sports leagues in the world. Since no transfer fees exist between different teams, most of the major financial decisions are about how much money a player should get paid per year. At the same time, a hard salary cap is applied to strictly constrain how much money a single team could spend. Therefore, making accurate estimations about players' salaries becomes a strategically critical problem that overpaying a player cripples a team's ability to sign other impactful players. Therefore, our research question is: **Can *we use the posterior predictive distribution to identify NBA players who are significantly overpaid or underpaid relative to their statistical market value?***

We apply Bayesian linear regression to explore the question, by merging 2025–2026 NBA player performance statistics with contracted salary data for 547 players with inner join, then fit two Stan-based Bayesian models, compare Hamiltonian Monte Carlo (HMC) against Variational Inference (VI), and generate 95% posterior predictive intervals to classify each player's pay status.

Our work differs from these existing references in three key aspects. First, we use the most recent 2025–2026 season data, reflecting the current salary cap and current players. Second, we construct full posterior predictive distributions, to make judgements about the fairness of individual salaries. Third, we compare the result of two posterior approximation strategies: HMC via Stan and mean-field Variational Inference, to determine which players are overpaid and which are underpaid.

2 Data

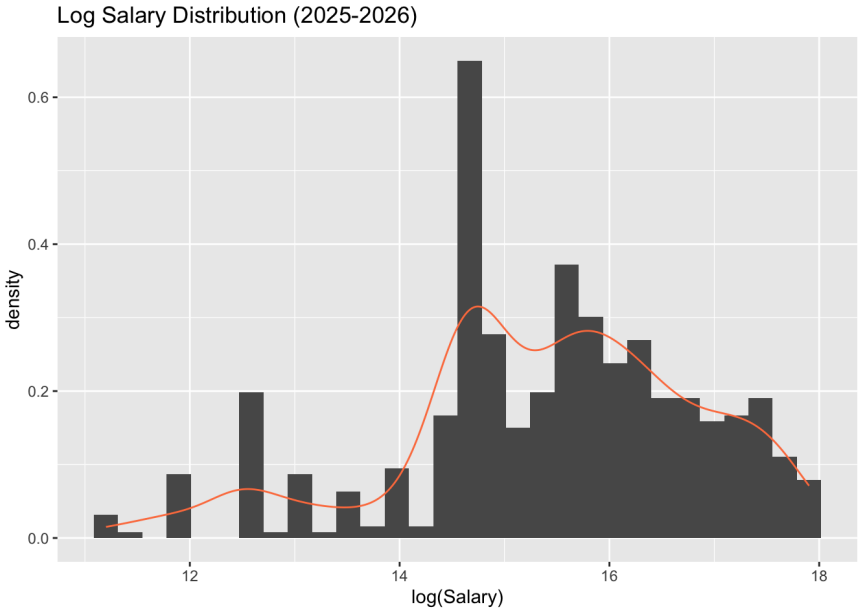
2.1 Dataset Source and Preprocessing

We merged two publicly available datasets: (1) the 2025–2026 NBA Player Statistics, containing per-game performance metrics for all players who appeared in the regular season. (2) the 2025–2026 NBA Player Contracts dataset, which records each player's salary for the current season. After an inner join on player name, the combined dataset contains 547 player-observations across 39 variables.

Salary columns were stored as character strings with a leading dollar sign, requiring conversion to numeric. One observation (Chris Livingston) had a missing current-year salary and was dropped, leaving **546 players**. Since there are some variables that are directly correlated with other variables (e.g. P+A is Points + Assists total per game), to maximize the preservation of original data, we chose the following performance predictors for modelling: Usage Rate (USG), Turnover Rate (TO), True Shooting Percentage (TS), Points per Game (PpG), Rebounds per Game (RpG), Assists per Game (ApG), Steals per Game (SpG), and Blocks per Game (BpG). All remaining contract-year columns and administrative identifiers were excluded.

2.2 Exploratory Data Analysis

The raw 2025–2026 salary distribution is heavily right-skewed, with the majority of players earning near the league minimum (~$263940) and a long tail extending to over $60M (e.g., Rudy Gobert at $35 M). Therefore a log transformation on salary(X2025.26) is applied to normalize skewed data.



Side by side boxplots of log(salary) stratified by position revealed that Forwards-Guards command the highest median salary, while Centers-Forwards show the most variability. After IQR-based outlier removal, the cleaned dataset contains 531 players across 7 positions.

We notice that for the correlation matrix, there's no correlation coefficient greater than 0.8, which means that all the variables at this stage are not considered to be highly correlated to each other. So we decided not to drop any of them.

Note that for our VIF values, there exists one VIF value of 6.016 (PpG) which is normally considered to be a concerning threshold that requires further investigation since this variable could potentially be affecting coefficient estimates. Since PpG represents points per game, higher PpG surely results in higher Salary. However, we decided not to drop this column, since this VIF value is not extremely high (less than 10), and this variable acts as a key factor to include for our research question.

All 8 predictors were standardised (zero mean, unit variance) before modelling to ensure that prior distributions are equally informative across variables and that posterior coefficients are directly comparable in magnitude.

3 Bayesian Models

3.1 Model 1: Bayesian Linear Regression

Our first model estimates the salary regardless of the player's position:

$$\log(salary)_i \sim \mathcal{N}(\alpha + \beta_1 \cdot USG_i + \beta_2 \cdot TO_i + \beta_3 \cdot TS_i + \beta_4 \cdot PpG_i + \beta_5 \cdot RpG_i + \beta_6 \cdot ApG_i + \beta_7 \cdot SpG_i + \beta_8 \cdot BpG_i,\ \sigma^2)$$

Prior specification:

$$\alpha \sim \mathcal{N}(16, 2); \beta_1, \beta_2, \beta_3, \beta_4, \beta_5, \beta_6, \beta_7, \beta_8 \sim \mathcal{N}(0, 1); \sigma \sim \text{Half-Normal}(0, 1)$$

The intercept prior N(16, 2) is centred near the log of the approximate median NBA salary and allows considerable uncertainty. The N(0, 1) coefficient priors are weakly informative on standardized predictors: they permit effect sizes up to roughly two log-salary units per standard deviation of a predictor, which is conservative relative to the data. We choose N(0,1) also because previously we scaled all the predictors. The Half-Normal(0, 1) prior on σ is appropriate given the OLS residual SD of 0.95.

3.2 Model 2: Hierarchical Bayesian Linear Regression

Our second model chose a different intercept assumption by allowing each of the seven positions to have its own baseline salary level, while still sharing the regression coefficients across positions:

$$\log(salary)_i \sim \mathcal{N}(\alpha_{j[i]} + \beta_1 \cdot USG_i + \beta_2 \cdot TO_i + \beta_3 \cdot TS_i + \beta_4 \cdot PpG_i + \beta_5 \cdot RpG_i + \beta_6 \cdot ApG_i + \beta_7 \cdot SpG_i + \beta_8 \cdot BpG_i,\ \sigma^2)$$

Hyperpriors and coefficient priors:

$$\alpha_j \sim \mathcal{N}(\mu_\alpha, \tau^2),\ j = 1, 2, ..., 7; \mu_\alpha \sim \mathcal{N}(16, 2); \tau \sim \text{Half-Normal}(0, 1); \beta_1, \beta_2, \beta_3, \beta_4, \beta_5, \beta_6, \beta_7, \beta_8 \sim \mathcal{N}(0, 1); \sigma \sim \text{Half-Normal}(0, 1)$$

The hierarchical structure is motivated by the EDA finding that position explains meaningful variation in log(salary). The partial-pooling property of the hierarchical prior means that positions with few observations borrow strength from the global distribution, reducing overfitting. The hyperparameter τ ~ Half-Normal(0, 1) allows the model to learn how much positions truly differ, rather than fixing this quantity a prior.

4 Results

4.1 Posterior Diagnostics

Both models were fitted using Stan's NUTS algorithm, which is based on HMC, with 4 chains of 4,000 iterations each (2000 warmup, 2000 post-warmup), yielding 8000 post-warmup draws. Both models achieved satisfactory convergence, with all R-hat values below 1.01 (Model 1 with 1.003, Model 2 with 1.001), indicating that the 4 chains mixed well. Effective sample sizes ranged from ~4270 to ~11448 across parameters, indicating efficient mixing.

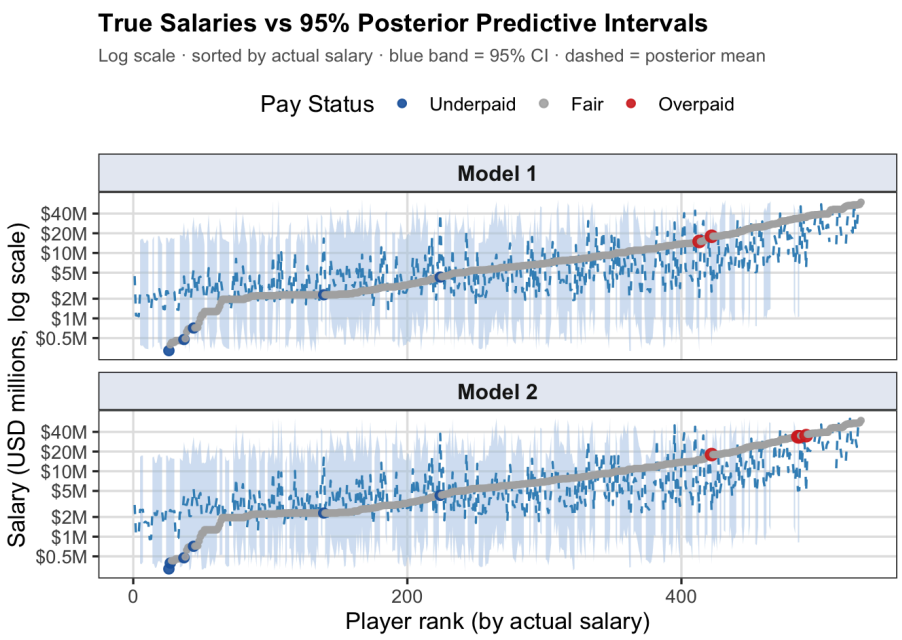
The dominant predictor in both models is PpG ($\beta_4 \approx$ 0.85 in Model 1, 0.81 in Model 2), confirming that points per game is the strongest predictor that influences salary. Turnover rate ($\beta_2 \approx 0.11 - 0.12$) shows a modest positive association. All other coefficients have 95% credible intervals around zero, indicating that their influences on our response variable is less important.

4.2 Model Comparison via LOO-CV

LOO-CV results show that Model 2 outperforms Model 1 with an ELPD difference of 6.6, and SE of 3.9, yielding a ratio of 1.69 standard errors. Although this falls slightly below the threshold of 2 for strong significance, Model 2 consistently achieves better predictive performance. This suggests that position-specific intercepts capture meaningful variation in salary structure, though the moderate improvement indicates that performance statistics, particularly points per game remain to be the dominant factor for player market value.

4.3 Posterior Predictive Salary Classification

For each player we extract the 95% posterior predictive interval from both fitted models. A player's actual salary is classified as overpaid, underpaid, and fair.



Model 1 identifies 2 overpaid players and 22 underpaid players. Model 2 identifies 4 overpaid players and 21 underpaid player-observations.

Several patterns are noticeable. We suspect many underpaid players are on minimum or two-way contracts despite performance metrics that show that the model associates with higher salaries. Or we suspect that these players who participated in very few games, that their on-court statistics in limited minutes do not translate to guaranteed value. For example, a player shown as underpaid reflects that the model's prediction is driven by his NBA statistics, while his actual contract reflects a minimum deal.

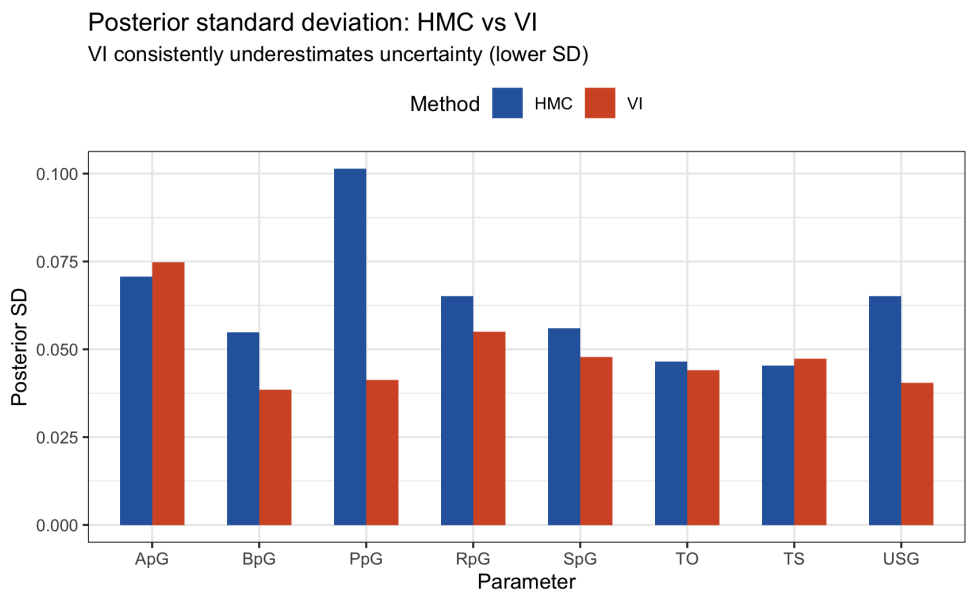
The 95% PPI plot (log scale, sorted by actual salary) shows that the predictive bands are wide enough to absorb most players, with only a small number of genuinely extreme outliers. This width is appropriate given σ ≈ 0.95 on the log scale, equivalent to a multiplicative factor of roughly $e^{1.86} \approx 6.4$ for the full 95% interval, reflecting the substantial residual noise in salary determination beyond statistical performance.

Figure shows the actual salaries of all 531 players against the 95% posterior predictive intervals from both models. The wide predictive intervals reflect substantial unexplained variation in NBA salaries beyond performance statistics. The majority of players fall within the predicted intervals, indicating adequate model calibration. A small number of players fall outside the 95% CI, those above the upper bound are classified as overpaid and those below the lower bound are classified as underpaid. Model 2 identifies slightly fewer outliers than Model 1, suggesting that accounting for position captures more variation.

5 HMC vs. Variational Inference

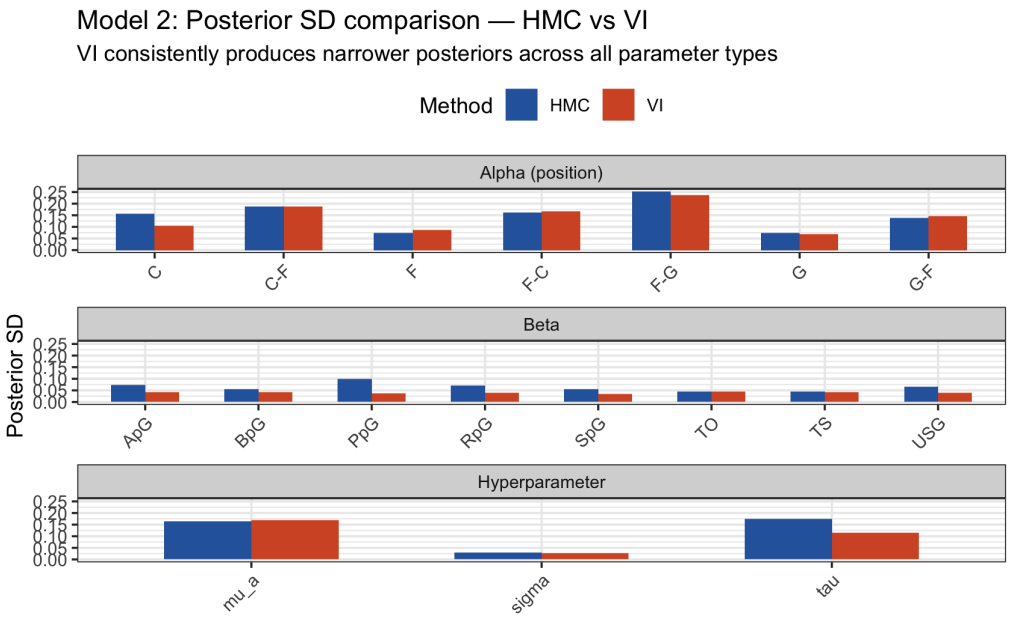
5.1 Variational Inference (Mean-Field)

The two tables compare posterior means and standard deviations between HMC and VI for Model 1 and Model 2 respectively. Across both models, posterior means are in close agreement, that the largest difference in Model 1 is 0.074 for TO, and in Model 2 is 0.016 for USG, both negligibly small relative to the parameter scales.



VI systematically underestimates posterior uncertainty for beta coefficients in both models, with PpG showing the most severe compression (sd_ratio = 0.407 in Model 1, 0.385 in Model 2). This is particularly consequential since PpG is the dominant predictor, we know that VI is overconfident about the parameter that matters most.

For Model 2's position-specific intercepts, the pattern is more mixed, only 3 out of 7 positions show VI underestimation, suggesting that the hierarchical model partially mitigates mean-field approximation error. However, the hyperparameter τ (sd_ratio = 0.661) is noticeably underestimated, meaning VI is overconfident about the degree of salary variation across positions. Based on these findings, we use HMC results for all subsequent inference.



6 Discussion & Conclusion

6.1 Key Findings

Our Bayesian analysis confirms that points per game is the main predictor of NBA salary. Note that our TO(turnover rate) shows a negative relationship with log(salary), which is counterintuitive but logically makes sense. Players who make more mistakes are paid more because high turnovers are often accompanied by high possession and high usage for basketball, and such players normally play a core role in the team. This shows that the NBA salary market does not punish mistakes, but rewards responsibility for holding the ball.

We find that most of the players are paid fairly, only a few players being identified as being significantly overpaid or underpaid. This finding is based on our 95% posterior predictive interval, where the majority of grey dots (represents actual salary) falls within the 95% interval. For those abnormal players, their salaries cannot be predicted by the NBA stats, there might exist an external factor such as two-ways contract, or injuries to explain such cases.

6.2 Limitations

Several flagged underpaid players appear in the dataset with very few games played. Their per game averages are unreliable small-sample estimates. The model has no mechanism to discount noisy observations. Future work should incorporate games played as a weight or model playing-time as a latent variable. Some players appear more than once in the dataset because they played for multiple teams in a single season. This introduces duplicated observations and inflated effective sample sizes for those players. A more principled treatment would aggregate statistics to the season level before modelling.

We only used the stats from the 2025-26 to predict the contract salaries for that same season. However, in reality, NBA contracts are usually based on a player's performance over several previous years. A player gets paid more due to their outstanding performance in previous seasons, but the statistics from this particular season may not reflect this.

Beyond that, since we have 7 positions in total, some positions have a lower number of players compared to that of other positions. This would cause imbalanced data, which means positions with fewer observations are less reliable and more uncertain.

Since our study is observational, we could only make the inference that PpG (points per game) are highly associated with salary, but we could not infer that increase in PpG results in the increase of salary.

6.3 Broader Context

Bayesian methods provide a clear and practical framework for salary valuation, helping teams assess not only whether a player is overpaid, but also the uncertainty and sensitivity behind that conclusion. The hierarchical model also provides a structured way to compare across position groups, it is useful when evaluating positional scarcity.

More broadly, this analysis illustrates how Bayesian posterior predictive checking can serve as an anomaly detection tool in any domain. The comparison between HMC and VI further highlights a practical trade-off, while VI is fast and accurate for estimating means, it tends to underestimate uncertainty, making it less reliable for tasks that depend on interval estimates and tail behavior.

7 Team Contributions

Desheng contributes mainly for the data processing, EDA, Stan code for two models, conducting the posterior predictive distributions. Yumeng contributes mainly for the model specifications (naive bayesian linear regression and a more complex hierarchical model), implementation variational inference for both models and final comparison visualizations. Both of us contribute in writing the final report.

References

Lichter, J., Wiemann, P. F. V., & Kneib, T. (2024). Variational inference: Uncertainty quantification in additive models. *AStA Advances in Statistical Analysis, 108*(2), 279–331. https://doi.org/10.1007/s10182-024-00492-4

NBAstuffer. (2026). *2025–2026 NBA player statistics dataset*. Retrieved April 11, 2026, from https://www.nbastuffer.com/2025-2026-nba-player-stats/

Ozbalta, E., Yavuz, M., & Kaya, T. (2021). Machine Learning: National Basketball Association Player Salary Prediction Using Supervised Machine Learning Methods. Intelligent and Fuzzy Techniques for Emerging Conditions and Digital Transformation *(*pp. 189–196). https://doi.org/10.1007/978-3-030-85577-2_22

Sports Reference LLC. (2026). *NBA player contracts dataset*. Basketball-Reference. Retrieved April 11, 2026, from https://www.basketball-reference.com/contracts/players.html

Appendix

Data Overview

```
library(tidyverse)
library(corrplot)
library(dplyr)
library(GGally)
library(car)

NBA_Player_Stats <- read.csv("Datasets/2025-2026 NBA Player Stats - NBAstuffer.csv")
NBA_Player_Stats <- NBA_Player_Stats[, -1]
NBA_Player_Contracts <- read.csv("Datasets/2025-26 NBA Player Contracts.csv")
result <- merge(NBA_Player_Stats, NBA_Player_Contracts, by.x = "NAME", by.y = "Player")
head(result)
```

```
          NAME TEAM CUR POS  AGE GP  MpG USG.  TO. FTA   FT. X2PA  X2P. X3PA
1    A.J. Green  Mil   *   G 26.5 60 29.5 13.3  9.7  41 0.805   46 0.500  418
2  Aaron Gordon  Den   *   F 30.5 26 27.5 22.9  7.8 121 0.793  183 0.557  114
3 Aaron Holiday  Hou   *   G 29.4 43 13.0 17.9 11.2  37 0.865   72 0.458  125
4 Aaron Nesmith  Ind   * G-F 26.4 37 29.6 19.7 10.5  97 0.825  192 0.427  227
5 Aaron Wiggins  Okc   *   G 27.2 51 23.5 19.8 12.7  66 0.742  223 0.502  231
6    Ace Bailey  Uta   *   F 19.6 58 27.1 20.1 10.6  62 0.742  375 0.517  282
   X3P.  eFG.   TS.  PpG RpG ApG SpG BpG TOpG  P.R  P.A P.R.A  VI  ORtg  DRtg
1 0.407 0.599 0.611  9.8 2.5 1.9 0.5 0.1  0.8 12.3 11.7  14.2 4.9 117.0 116.3
2 0.386 0.566 0.617 16.6 6.1 2.5 0.7 0.2  1.0 22.7 19.1  25.2 9.2 128.1 113.8
3 0.360 0.510 0.546  5.4 0.9 0.9 0.5 0.1  0.6  6.3  6.3   7.2 5.0 114.5 108.7
4 0.352 0.482 0.524 13.1 4.5 2.1 0.6 0.6  1.3 17.5 15.2  19.6 6.7 106.0 114.0
5 0.355 0.518 0.537 10.2 3.4 1.8 0.9 0.5  1.3 13.5 12.0  15.4 6.8 104.6 106.4
6 0.344 0.517 0.530 12.5 4.0 1.7 0.9 0.5  1.3 16.5 14.2  18.2 6.5 106.1 116.9
  Rank Team  X2025.26  X2026.27  X2027.28  X2028.29  X2029.30 X2030.31
1  355  MIL  $2301587 $10044644 $10848215 $11651786 $12455355
2   82  DEN $22841455 $33658037 $36350680 $39043323
3  386  HOU  $2296274
4  158  IND $11000000 $11000000 $19418000 $20971440
5  172  OKC $10102803  $9224300  $8345797  $8345797
6  184  UTA  $9069840  $9523080  $9976560 $12640302
  Guaranteed Additional
1  $47301587  greenaj01
2  $92850172  gordoaa01
3   $2296274  holidaa01
4  $62389440  nesmiaa01
5  $27672900  wiggiaa01
6  $18592920  baileac01
```

```
dim(result)
```

```
[1] 547  39
```

Data Cleaning

We notice that our contracted salary columns (X2025.26 to X2030.31) are treated as character instead of numeric because of the extra dollar sign ($). We must remove the sign and set these to numeric for later analysis.

```
result <- result %>%
  mutate(across(starts_with("X20"), ~ as.numeric(gsub("\\$", "", .x))))
class(result$X2025.26)
```

```
[1] "numeric"
```

We also need to check whether there are any NA values for our response variable; if NA exists we would need to drop that row.

```
result[is.na(result$X2025.26), ]
```

```
               NAME TEAM CUR POS  AGE GP MpG USG. TO. FTA FT. X2PA X2P. X3PA
88 Chris Livingston  Cle   *   F 22.4  3 5.8 17.9   0   1   1    4    1    3
   X3P.  eFG.   TS. PpG RpG ApG SpG BpG TOpG P.R P.A P.R.A  VI ORtg DRtg Rank
```

```
88    0 0.571 0.605   3   1 0.3 0.3   0    0   4 3.3   4.3 6.7  129   91  381
   Team X2025.26 X2026.27 X2027.28 X2028.29 X2029.30 X2030.31 Guaranteed
88  MIL       NA       NA       NA       NA       NA       NA   $2296274
   Additional
88  livinch01
```

There exists one row which misses the value for our response variable, we drop this observation.

```
result_final <- result[!is.na(result$X2025.26), ]
dim(result_final)
```

```
[1] 546  39
```

Next, we select the relevant columns for our research question and rename them for clarity.

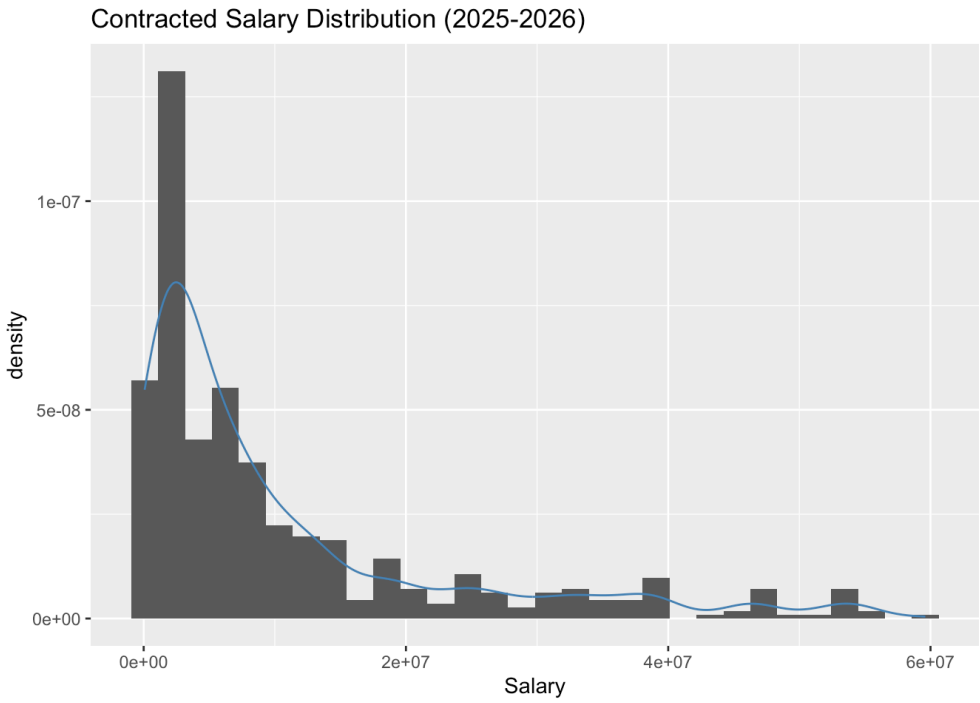
```
NBA_Player_Stats_clean <- result_final[, c("NAME", "TEAM", "CUR", "POS", "AGE", "GP", "MpG", "USG.", "TO.", "TS.", "

df <- NBA_Player_Stats_clean
df <- rename(df, Salary = X2025.26)   # rename salary
names(df) <- sub("\\..*", "", names(df))  # remove dots in names
head(df)
```

```
           NAME TEAM CUR POS  AGE GP  MpG  USG   TO    TS  PpG RpG ApG SpG BpG
1    A.J. Green  Mil   *   G 26.5 60 29.5 13.3  9.7 0.611  9.8 2.5 1.9 0.5 0.1
2  Aaron Gordon  Den   *   F 30.5 26 27.5 22.9  7.8 0.617 16.6 6.1 2.5 0.7 0.2
3 Aaron Holiday  Hou   *   G 29.4 43 13.0 17.9 11.2 0.546  5.4 0.9 0.9 0.5 0.1
4 Aaron Nesmith  Ind   * G-F 26.4 37 29.6 19.7 10.5 0.524 13.1 4.5 2.1 0.6 0.6
5 Aaron Wiggins  Okc   *   G 27.2 51 23.5 19.8 12.7 0.537 10.2 3.4 1.8 0.9 0.5
6    Ace Bailey  Uta   *   F 19.6 58 27.1 20.1 10.6 0.530 12.5 4.0 1.7 0.9 0.5
    Salary
1  2301587
2 22841455
3  2296274
4 11000000
5 10102803
6  9069840
```

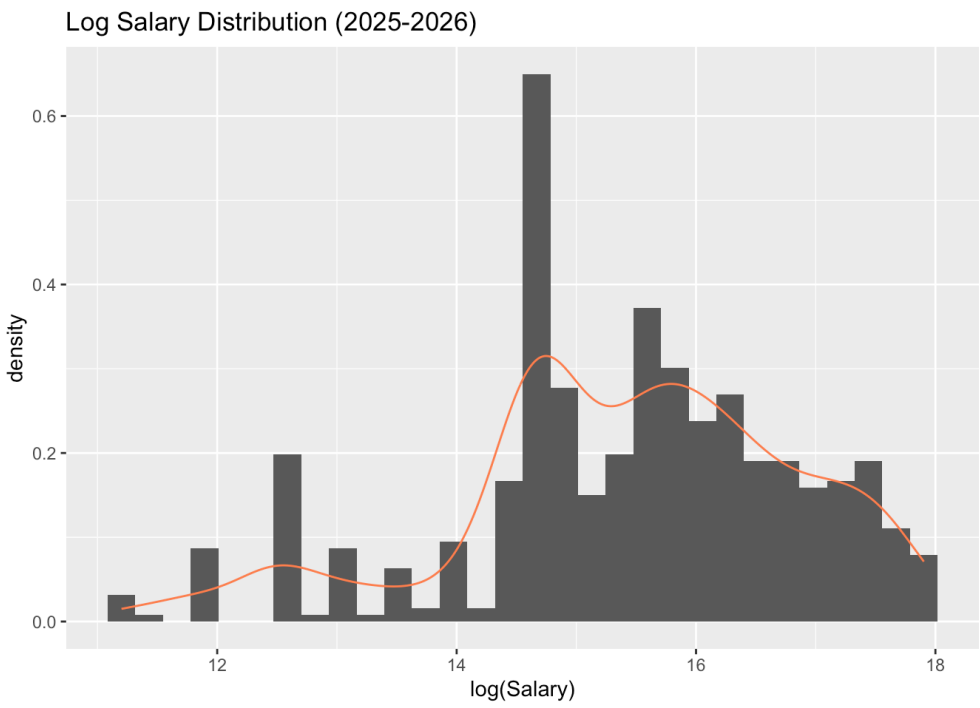
EDA

Salary Distribution

```
# Original Salary
p1 <- ggplot(df, aes(x = Salary)) +
  geom_histogram(aes(y = after_stat(density)), bins = 30) +
  geom_density(color = "steelblue") +
  labs(title = "Contracted Salary Distribution (2025-2026)")

p1
```



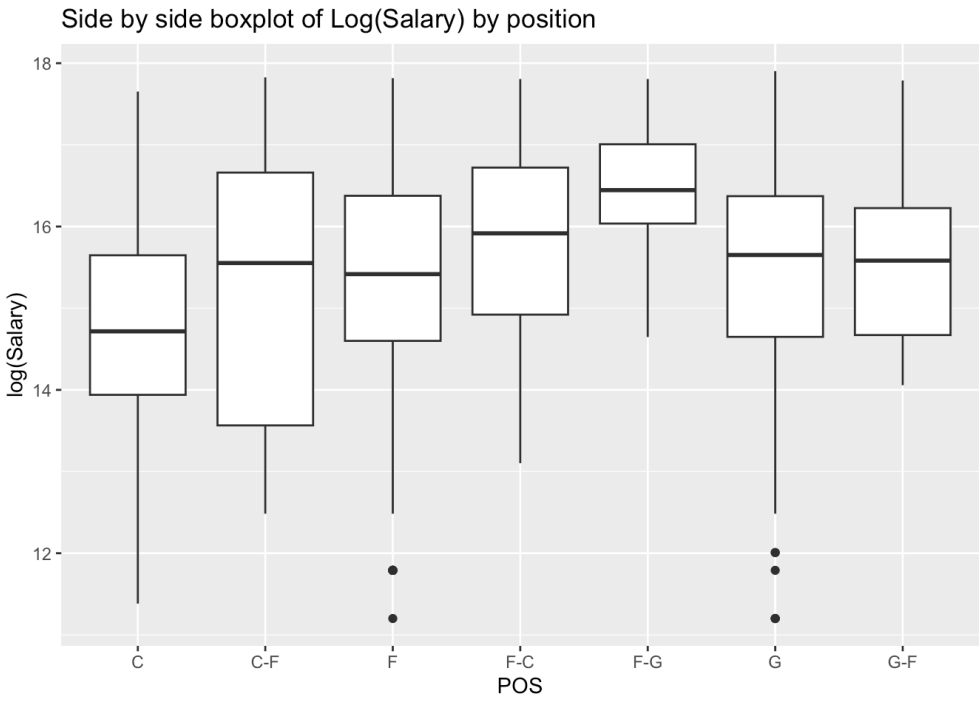

```
# Log Salary
p2 <- ggplot(df, aes(x = log(Salary))) +
  geom_histogram(aes(y = after_stat(density)), bins = 30) +
  geom_density(color = "coral") +
  labs(title = "Log Salary Distribution (2025-2026)")

p2
```



We notice that the distribution of the contracted salary (X2025.26) is heavily right-skewed. A log transformation is necessary in order to standardize the variable.

Salary by Position

```
# Position
ggplot(df, aes(x = POS, y = log(Salary))) +
  geom_boxplot() +
  labs(title = "Side by side boxplot of Log(Salary) by position")
```

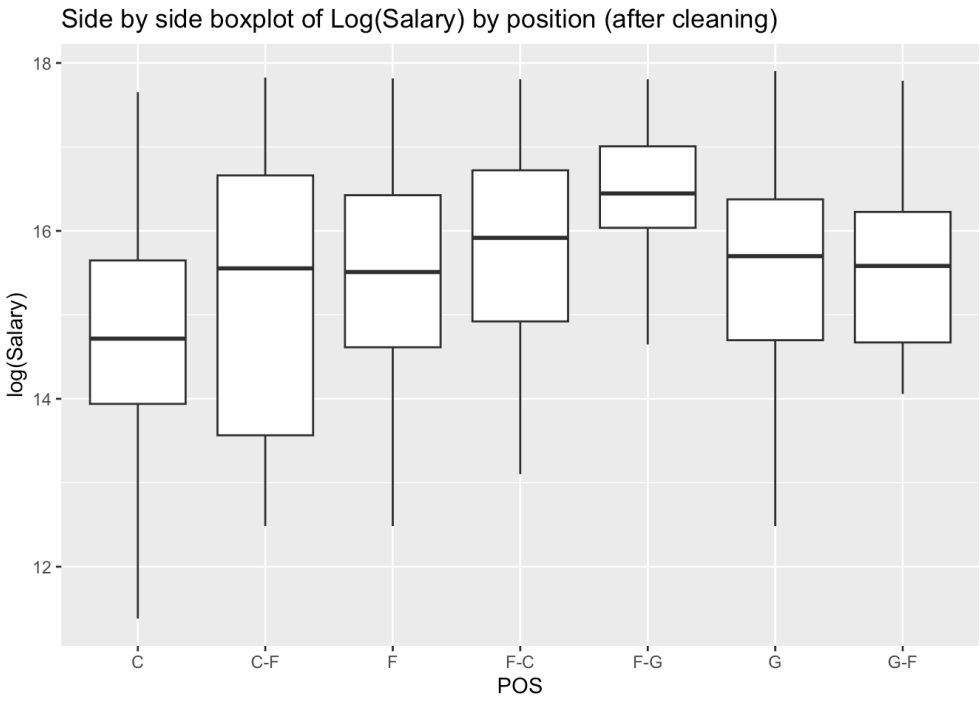


Outliers Removal

```
# remove outliers
df <- df %>%
  group_by(POS) %>%
  mutate(
    Q1 = quantile(log(Salary), 0.25, na.rm = TRUE),
    Q3 = quantile(log(Salary), 0.75, na.rm = TRUE),
    IQR = Q3 - Q1,
    lower = Q1 - 1.5 * IQR
  ) %>%
  filter(!(POS %in% c("F", "G") & log(Salary) < lower)) %>%
  select(-Q1, -Q3, -IQR, -lower) %>%
  ungroup()
dim(df)
```

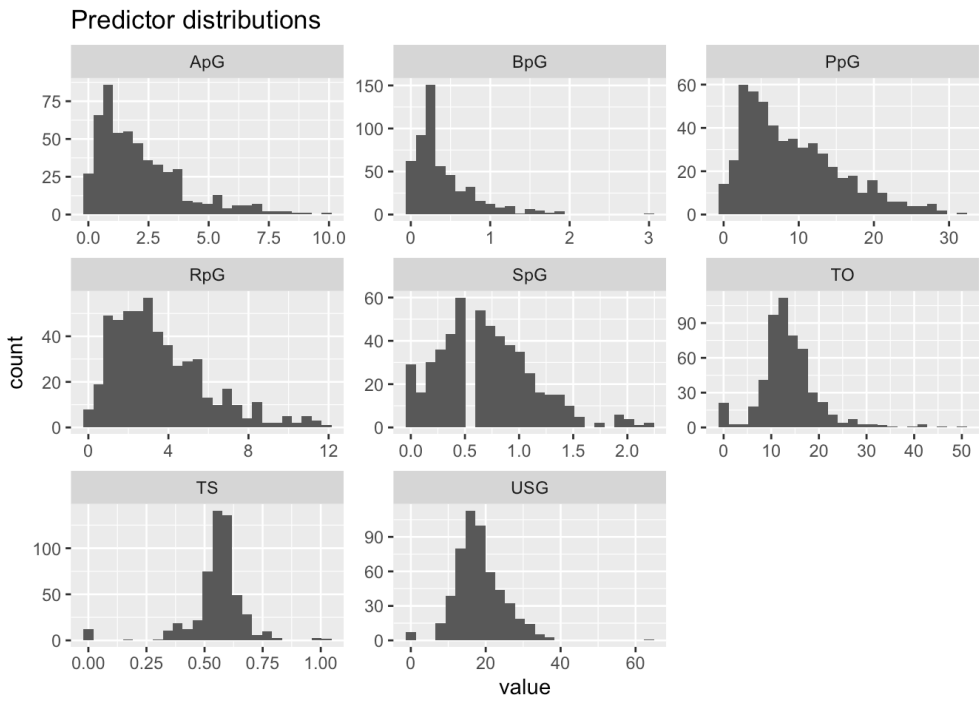
```
[1] 531  16
```

```
# Position boxplot after data cleaning
ggplot(df, aes(x = POS, y = log(Salary))) +
  geom_boxplot() +
  labs(title = "Side by side boxplot of Log(Salary) by position (after cleaning)")
```



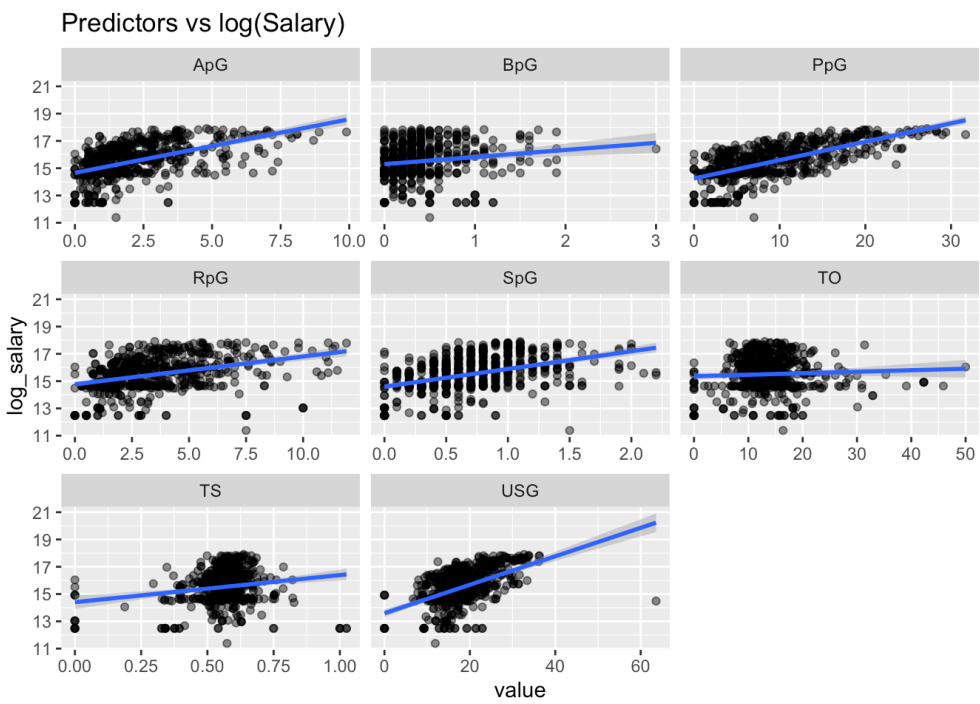
Predictor Distributions and Relationships with Log(Salary)

```
pred_vars <- c("USG", "TO", "TS", "PpG", "RpG", "ApG", "SpG", "BpG")

# Histogram
df %>%
  select(all_of(pred_vars)) %>%
  pivot_longer(everything()) %>%
  ggplot(aes(x = value)) +
  geom_histogram(bins = 25) +
  facet_wrap(~name, scales = "free") +
  labs(title = "Predictor distributions")
```



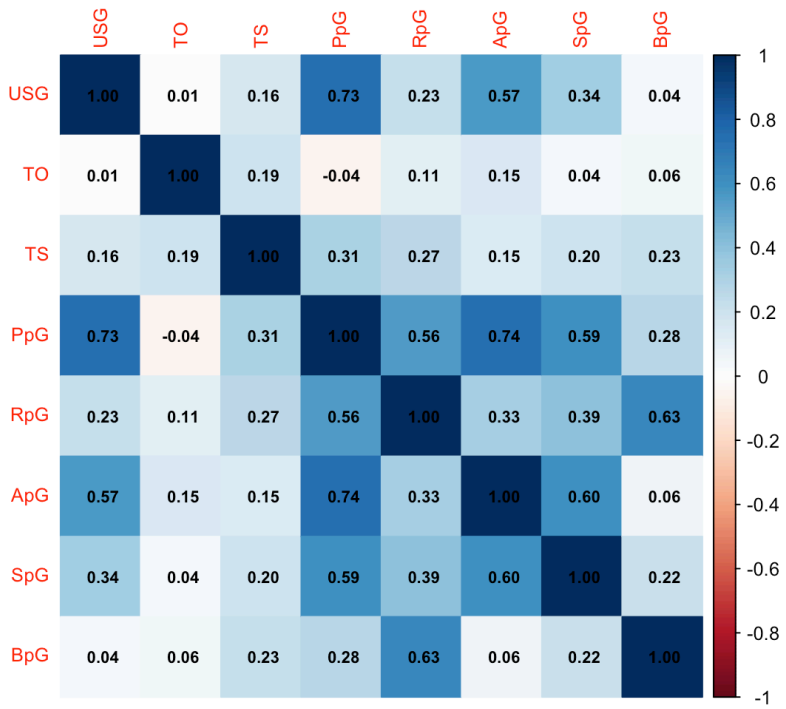

```
# Variables vs log(Salary) Scatterplot
df %>%
  mutate(log_salary = log(Salary)) %>%
  select(log_salary, all_of(pred_vars)) %>%
```

```
  pivot_longer(-log_salary) %>%
  ggplot(aes(x = value, y = log_salary)) +
  geom_point(alpha = 0.5) +
  geom_smooth(method = "lm", se = TRUE) +
  facet_wrap(~name, scales = "free_x") +
  labs(title = "Predictors vs log(Salary)")
```



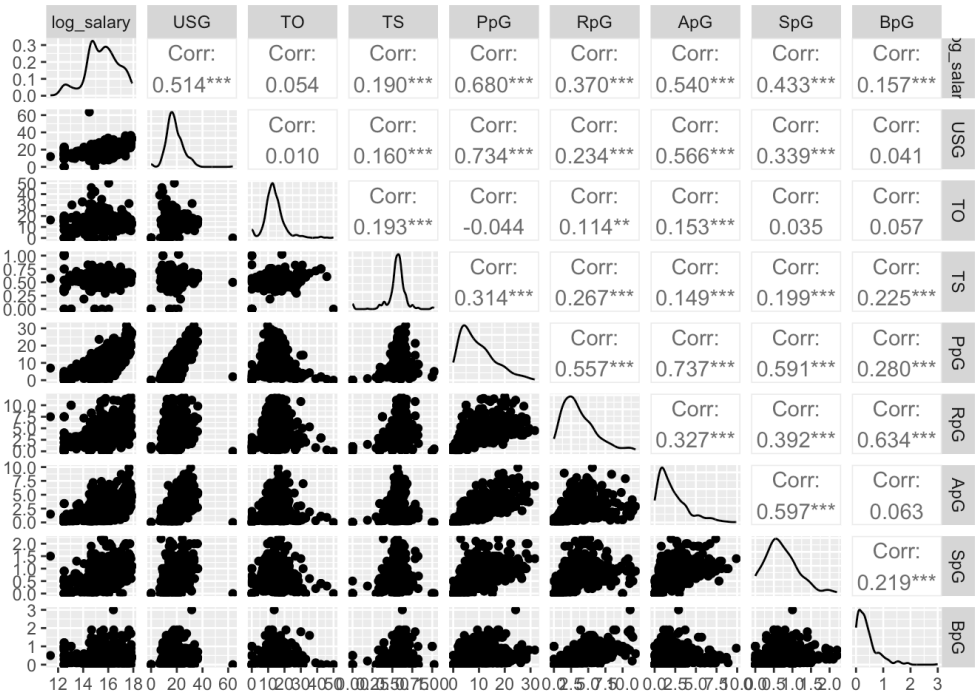
Correlation and Multicollinearity Check

```
# Correlation Plot / Matrix
cor_mat <- cor(df[, pred_vars], use = "complete.obs")
corrplot::corrplot(cor_mat, method = "color", addCoef.col = "black",
                   number.cex = 0.7, tl.cex = 0.8)
```




```
# Pairs plot (log Salary)
df %>%
  mutate(log_salary = log(Salary)) %>%
```

```
  select(log_salary, all_of(pred_vars)) %>%
  ggpairs()
```




```
# VIF
ols_fit <- lm(log(Salary) ~ USG + TO + TS + PpG + RpG + ApG + SpG + BpG,
              data = df)
vif(ols_fit)
```

```
     USG       TO       TS      PpG      RpG      ApG      SpG      BpG
2.543840 1.244512 1.239870 6.016229 2.464104 2.920397 1.777892 1.773535
```

Scaling for Standardizing Variables

```
df <- df %>%
  mutate(
    log_salary = log(Salary),
    USG_s = scale(USG)[,1],
    TO_s  = scale(TO)[,1],
    TS_s  = scale(TS)[,1],
    PpG_s = scale(PpG)[,1],
    RpG_s = scale(RpG)[,1],
    ApG_s = scale(ApG)[,1],
    SpG_s = scale(SpG)[,1],
    BpG_s = scale(BpG)[,1]
  )
head(df)
```

```
# A tibble: 6 × 25
  NAME   TEAM  CUR   POS     AGE    GP   MpG   USG    TO    TS   PpG   RpG   ApG
  <chr>  <chr> <chr> <chr> <dbl> <int> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1 A.J. … Mil   *     G      26.5    60  29.5  13.3   9.7 0.611   9.8   2.5   1.9
2 Aaron… Den   *     F      30.5    26  27.5  22.9   7.8 0.617  16.6   6.1   2.5
3 Aaron… Hou   *     G      29.4    43  13    17.9  11.2 0.546   5.4   0.9   0.9
4 Aaron… Ind   *     G-F    26.4    37  29.6  19.7  10.5 0.524  13.1   4.5   2.1
5 Aaron… Okc   *     G      27.2    51  23.5  19.8  12.7 0.537  10.2   3.4   1.8
6 Ace B… Uta   *     F      19.6    58  27.1  20.1  10.6 0.53   12.5   4     1.7
# ℹ 12 more variables: SpG <dbl>, BpG <dbl>, Salary <dbl>, log_salary <dbl>,
#   USG_s <dbl>, TO_s <dbl>, TS_s <dbl>, PpG_s <dbl>, RpG_s <dbl>, ApG_s <dbl>,
#   SpG_s <dbl>, BpG_s <dbl>
```

Model 1: Bayesian Linear Regression

Our naive model is linear regression.

$$\log(salary)_i \sim \mathcal{N}(\alpha + \beta_1 \cdot USG_i + \beta_2 \cdot TO_i + \beta_3 \cdot TS_i + \beta_4 \cdot PpG_i + \beta_5 \cdot RpG_i + \beta_6 \cdot ApG_i + \beta_7 \cdot SpG_i + \beta_8 \cdot BpG_i,\ \sigma^2)$$

And we have

$$\alpha \sim \mathcal{N}(16, 2)\beta_1, \beta_2, \beta_3, \beta_4, \beta_5, \beta_6, \beta_7, \beta_8 \sim \mathcal{N}(0, 1)\sigma \sim \text{Half-Normal}(0, 1)$$

Model 2: Hierarchical Bayesian Linear Regression

Our complex model is Hierarchical Linear Regression with position-specific intercepts.

$$\log(salary)_i \sim \mathcal{N}(\alpha_{j[i]} + \beta_1 \cdot USG_i + \beta_2 \cdot TO_i + \beta_3 \cdot TS_i + \beta_4 \cdot PpG_i + \beta_5 \cdot RpG_i + \beta_6 \cdot ApG_i + \beta_7 \cdot SpG_i + \beta_8 \cdot BpG_i,\ \sigma^2)$$

Where $j[i]$ represents the i-th basketball player's position, $j = 1, 2, \ldots, 7$, that we have different 7 positions in total.

$$\alpha_j \sim \mathcal{N}(\mu_\alpha, \tau^2),\ j = 1, 2, \ldots, 7\mu_\alpha \sim \mathcal{N}(16, 2)\tau \sim \text{Half-Normal}(0, 1)\beta_1, \beta_2, \beta_3, \beta_4, \beta_5, \beta_6, \beta_7, \beta_8 \sim \mathcal{N}(0, 1)\sigma \sim \text{Half-Normal}(0, 1)$$

Run Posterior Using Stan

```
library(rstan)
library(loo)

options(mc.cores = parallel::detectCores())
rstan_options(auto_write = TRUE)

pred_vars <- c("USG", "TO", "TS", "PpG", "RpG", "ApG", "SpG", "BpG")

# standardize
X_raw <- df[, pred_vars]
X_scaled <- scale(X_raw)

y <- log(df$Salary)

# position index
pos_levels <- levels(factor(df$POS))
pos_id     <- as.integer(factor(df$POS, levels = pos_levels))
J          <- length(pos_levels)
N          <- nrow(df)
K          <- length(pred_vars)

cat("N =", N, "| K =", K, "| J =", J, "\n")
```

```
N = 531 | K = 8 | J = 7
```

```
cat("Positions:", pos_levels, "\n")
```

```
Positions: C C-F F F-C F-G G G-F
```

```
# model 1 data
stan_data_m1 <- list(
  N = N,
  K = K,
  X = X_scaled,
  y = y
)

fit_m1 <- stan(
  file    = "model1.stan",
  data    = stan_data_m1,
  iter    = 4000,          # 2000 warmup + 2000 sampling
  warmup  = 2000,
  chains  = 4,
  seed    = 42,
  control = list(adapt_delta = 0.95),
  refresh = 0
)
```

```r
print(fit_m1, pars = c("alpha", "beta", "sigma"), probs = c(0.025, 0.5, 0.975))
```

```
Inference for Stan model: anon_model.
4 chains, each with iter=4000; warmup=2000; thin=1;
post-warmup draws per chain=2000, total post-warmup draws=8000.

         mean se_mean   sd  2.5%   50% 97.5% n_eff Rhat
alpha   15.51       0 0.04 15.43 15.51 15.59 11448    1
beta[1]  0.02       0 0.07 -0.11  0.02  0.15  6534    1
beta[2]  0.11       0 0.05  0.02  0.11  0.21  7620    1
beta[3] -0.05       0 0.05 -0.14 -0.05  0.04  8802    1
beta[4]  0.85       0 0.10  0.65  0.85  1.05  4270    1
beta[5]  0.00       0 0.07 -0.12  0.00  0.13  7132    1
beta[6]  0.02       0 0.07 -0.11  0.03  0.16  7097    1
beta[7]  0.05       0 0.06 -0.06  0.05  0.17 11239    1
beta[8] -0.05       0 0.05 -0.15 -0.04  0.06  8426    1
sigma    0.95       0 0.03  0.90  0.95  1.01 10538    1

Samples were drawn using NUTS(diag_e) at Sat Apr 11 22:40:41 2026.
For each parameter, n_eff is a crude measure of effective sample size,
and Rhat is the potential scale reduction factor on split chains (at
convergence, Rhat=1).
```

```r
# model 2 data
stan_data_m2 <- list(
  N      = N,
  K      = K,
  J      = J,
  X      = X_scaled,
  y      = y,
  pos_id = pos_id
)

fit_m2 <- stan(
  file    = "model2.stan",
  data    = stan_data_m2,
  iter    = 4000,
  warmup  = 2000,
  chains  = 4,
  seed    = 42,
  control = list(adapt_delta = 0.95),
  refresh = 0
)

print(fit_m2, pars = c("mu_a", "tau", "alpha", "beta", "sigma"),
      probs = c(0.025, 0.5, 0.975))
```

```
Inference for Stan model: anon_model.
4 chains, each with iter=4000; warmup=2000; thin=1;
post-warmup draws per chain=2000, total post-warmup draws=8000.

          mean se_mean   sd  2.5%   50% 97.5% n_eff Rhat
mu_a     15.53       0 0.16 15.21 15.53 15.87  5221    1
tau       0.37       0 0.17  0.14  0.34  0.80  4132    1
alpha[1] 15.09       0 0.16 14.78 15.09 15.39  4889    1
alpha[2] 15.44       0 0.19 15.06 15.44 15.79  5976    1
alpha[3] 15.58       0 0.08 15.43 15.58 15.73  8949    1
alpha[4] 15.75       0 0.16 15.45 15.75 16.08  7111    1
alpha[5] 15.93       0 0.25 15.47 15.92 16.46  5810    1
alpha[6] 15.53       0 0.07 15.39 15.53 15.68  7018    1
alpha[7] 15.43       0 0.14 15.17 15.44 15.70  8311    1
beta[1]   0.02       0 0.06 -0.11  0.02  0.15  6073    1
beta[2]   0.12       0 0.05  0.03  0.12  0.21  6890    1
beta[3]  -0.06       0 0.05 -0.15 -0.06  0.03  7553    1
beta[4]   0.81       0 0.10  0.61  0.81  1.00  4322    1
beta[5]   0.05       0 0.07 -0.09  0.05  0.19  5612    1
beta[6]   0.03       0 0.07 -0.11  0.03  0.18  5984    1
beta[7]   0.03       0 0.06 -0.08  0.03  0.14  7190    1
beta[8]  -0.01       0 0.06 -0.12 -0.01  0.10  6149    1
sigma     0.94       0 0.03  0.88  0.94  1.00  9435    1
```

```
Samples were drawn using NUTS(diag_e) at Sat Apr 11 22:41:14 2026.
For each parameter, n_eff is a crude measure of effective sample size,
and Rhat is the potential scale reduction factor on split chains (at
convergence, Rhat=1).
```

```r
stan_diag_m1 <- summary(fit_m1)$summary
stan_diag_m2 <- summary(fit_m2)$summary

cat("\nModel 1 - max R-hat:", max(stan_diag_m1[, "Rhat"], na.rm = TRUE), "\n")
```

```
Model 1 - max R-hat: 1.003142
```

```r
cat("Model 2 - max R-hat:", max(stan_diag_m2[, "Rhat"], na.rm = TRUE), "\n")
```

```
Model 2 - max R-hat: 1.00099
```

LOO-CV for Result Comparison

```r
# leave one out cross validation
log_lik_m1 <- extract_log_lik(fit_m1, merge_chains = FALSE)
log_lik_m2 <- extract_log_lik(fit_m2, merge_chains = FALSE)

loo_m1 <- loo(log_lik_m1, r_eff = relative_eff(exp(log_lik_m1)))
loo_m2 <- loo(log_lik_m2, r_eff = relative_eff(exp(log_lik_m2)))

loo_compare(loo_m1, loo_m2)
```

```
       elpd_diff se_diff
model2  0.0       0.0
model1 -6.6       3.9
```

```r
cat("\nLoo compare:", loo_compare(loo_m1, loo_m2), "\n")
```

```
Loo compare: 0 -6.588733 0 3.934333 -726.4517 -733.0404 16.97249 16.99469 15.6422 11.01665 1.42604 1.09894 1452.903
1466.081 33.94497 33.98937
```

95% Posterior Predictive Interval of Results

```r
# Model 1
y_rep_model1  <- extract(fit_m1, "y_rep")$y_rep

# posterior predictive distribution mean & 95% ci (log scale)
ppd_model1_mean  <- apply(y_rep_model1, 2, mean)
ppd_model1_lower <- apply(y_rep_model1, 2, quantile, 0.025)
ppd_model1_upper <- apply(y_rep_model1, 2, quantile, 0.975)

results_model1 <- data.frame(
  NAME          = df$NAME,
  POS           = df$POS,
  Actual_Salary = df$Salary,
  log_Salary    = y,
  PPD_mean      = ppd_model1_mean,
  PPD_lower     = ppd_model1_lower,
  PPD_upper     = ppd_model1_upper,
  # change back to original salary
  Predicted_Salary = exp(ppd_model1_mean),
  Lower_Salary     = exp(ppd_model1_lower),
  Upper_Salary     = exp(ppd_model1_upper)
)

# overpaid / underpaid
results_model1$Status <- "Fair"
results_model1$Status[results_model1$log_Salary > results_model1$PPD_upper] <- "Overpaid"
results_model1$Status[results_model1$log_Salary < results_model1$PPD_lower] <- "Underpaid"
```

```r
cat("\nOverpaid players:\n")
```

```
Overpaid players:
```

```r
print(results_model1[results_model1$Status == "Overpaid",
              c("NAME", "POS", "Actual_Salary", "Predicted_Salary")])
```

```
             NAME POS Actual_Salary Predicted_Salary
271 Jonathan Isaac   F      15000000          2121136
314  Kevin Huerter G-F      17991071          2563025
```

```r
cat("\nUnderpaid players:\n")
```

```
Underpaid players:
```

```r
print(results_model1[results_model1$Status == "Underpaid",
              c("NAME", "POS", "Actual_Salary", "Predicted_Salary")])
```

```
                  NAME POS Actual_Salary Predicted_Salary
81       Charles Bassey C-F        263940          2460660
82       Charles Bassey C-F        263940          2424952
83       Charles Bassey C-F        263940          2451396
89     Christian Koloko   C        263940          2275392
90     Christian Koloko   C        263940          2237604
91     Christian Koloko   C        263940          2257355
92     Christian Koloko   C        263940          2224512
111       Dalano Banton   F        263940          2500621
112       Dalano Banton   F        263940          2533152
113       Dalano Banton   F        263940          1970704
114       Dalano Banton   F        263940          1941259
280       Jordan Miller   G        712637          5544594
319      Keyonte George   G       4278960         40784914
323       Killian Hayes   G        263940          2776887
324       Killian Hayes   G        263940          2799348
332        Kobe Sanders   G        475497          4378675
347     Lawson Lovering   C         87784          4427480
394        Monte Morris   G        321184          2182365
431 Patrick Baldwin Jr.   F        263940          2204641
432 Patrick Baldwin Jr.   F        263940          2138125
435 Patrick Baldwin Jr.   F        263940          2603725
436 Patrick Baldwin Jr.   F        263940          2597286
458   Russell Westbrook   G       2296274         16629512
```

```r
# Model 2
y_rep_model2  <- extract(fit_m2, "y_rep")$y_rep

# posterior predictive distribution mean & 95% ci (log scale)
ppd_model2_mean  <- apply(y_rep_model2, 2, mean)
ppd_model2_lower <- apply(y_rep_model2, 2, quantile, 0.025)
ppd_model2_upper <- apply(y_rep_model2, 2, quantile, 0.975)

results_model2 <- data.frame(
  NAME          = df$NAME,
  POS           = df$POS,
  Actual_Salary = df$Salary,
  log_Salary    = y,
  PPD_mean      = ppd_model2_mean,
  PPD_lower     = ppd_model2_lower,
  PPD_upper     = ppd_model2_upper,
  # change back to original salary
  Predicted_Salary = exp(ppd_model2_mean),
  Lower_Salary     = exp(ppd_model2_lower),
  Upper_Salary     = exp(ppd_model2_upper)
)

# overpaid / underpaid
results_model2$Status <- "Fair"
```

```r
results_model2$Status[results_model2$log_Salary > results_model2$PPD_upper] <- "Overpaid"
results_model2$Status[results_model2$log_Salary < results_model2$PPD_lower] <- "Underpaid"

cat("\nOverpaid players:\n")
```

```
Overpaid players:
```

```r
print(results_model2[results_model2$Status == "Overpaid",
              c("NAME", "POS", "Actual_Salary", "Predicted_Salary")])
```

```
            NAME POS Actual_Salary Predicted_Salary
130 Deandre Ayton   C      33654814          5063748
131 Deandre Ayton   C      33654814          5143064
314 Kevin Huerter G-F      17991071          2456192
456   Rudy Gobert   C      35000000          4837245
```

```r
cat("\nUnderpaid players:\n")
```

```
Underpaid players:
```

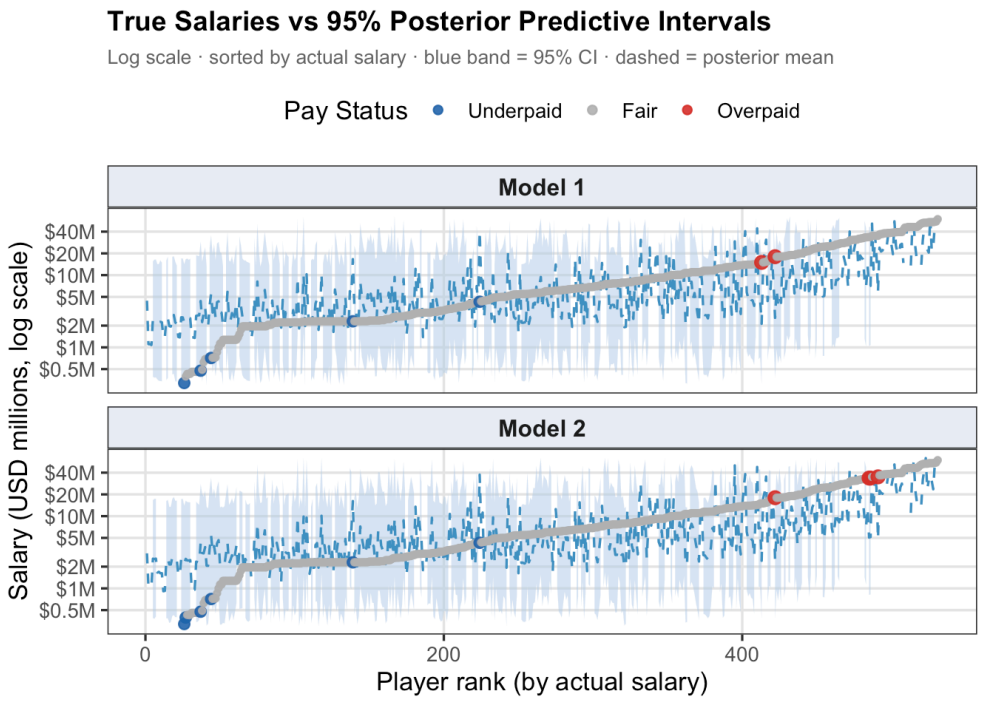
```r
print(results_model2[results_model2$Status == "Underpaid",
              c("NAME", "POS", "Actual_Salary", "Predicted_Salary")])
```

```
                  NAME POS Actual_Salary Predicted_Salary
81       Charles Bassey C-F        263940          2575444
82       Charles Bassey C-F        263940          2550302
83       Charles Bassey C-F        263940          2581074
90     Christian Koloko   C        263940          1694604
111       Dalano Banton   F        263940          2529156
112       Dalano Banton   F        263940          2518943
113       Dalano Banton   F        263940          2146119
114       Dalano Banton   F        263940          2147026
280       Jordan Miller   G        712637          5451340
319      Keyonte George   G       4278960         37488205
323       Killian Hayes   G        263940          2723548
324       Killian Hayes   G        263940          2744729
332        Kobe Sanders   G        475497          4283779
347     Lawson Lovering   C         87784          3052961
394        Monte Morris   G        321184          2242864
399       Myron Gardner   F        395029          2561054
431 Patrick Baldwin Jr.   F        263940          2563822
432 Patrick Baldwin Jr.   F        263940          2511565
435 Patrick Baldwin Jr.   F        263940          2630335
436 Patrick Baldwin Jr.   F        263940          2581986
458   Russell Westbrook   G       2296274         16152846
```

True salaries against the 95% posterior predictive intervals Plot

```r
library(ggplot2)
library(dplyr)
library(scales)

make_plot_df <- function(res, model_label) {
  res %>%
    arrange(Actual_Salary) %>%
    mutate(
      player_idx = row_number(),
      Model      = model_label,
      Status     = factor(Status, levels = c("Underpaid", "Fair", "Overpaid"))
    )
}

plot_df1 <- make_plot_df(results_model1, "Model 1")
plot_df2 <- make_plot_df(results_model2, "Model 2")
plot_df  <- bind_rows(plot_df1, plot_df2)

status_colours <- c(
  Underpaid = "#2166ac",
```

```r
  Fair      = "#b0b0b0",
  Overpaid  = "#d73027"
)

salary_cap <- quantile(plot_df$Upper_Salary, 0.97) / 1e6  # clip top 3%

ggplot(plot_df, aes(x = player_idx)) +

  # 95 % posterior predictive interval ribbon
  geom_ribbon(
    aes(
      ymin = pmax(Lower_Salary / 1e6, 0.01),
      ymax = pmin(Upper_Salary / 1e6, salary_cap)
    ),
    fill = "#a8c8e8", alpha = 0.45
  ) +

  # Predicted salary line
  geom_line(
    aes(y = pmin(Predicted_Salary / 1e6, salary_cap)),
    colour = "#4393c3", linewidth = 0.6, linetype = "dashed"
  ) +

  # Actual salary dots, coloured by pay status
  geom_point(
    aes(y = Actual_Salary / 1e6, colour = Status, size = Status),
    alpha = 0.9
  ) +

  scale_colour_manual(values = status_colours, name = "Pay Status") +
  scale_size_manual(
    values = c(Underpaid = 2.2, Fair = 1.2, Overpaid = 2.8),
    guide = "none"
  ) +

  scale_y_log10(
    labels = function(x) paste0("$", round(x, 1), "M"),
    breaks = c(0.5, 1, 2, 5, 10, 20, 40),
    limits = c(0.3, max(plot_df$Actual_Salary / 1e6) * 1.1)
  ) +

  facet_wrap(~Model, ncol = 1) +

  labs(
    title    = "True Salaries vs 95% Posterior Predictive Intervals",
    subtitle = "Log scale · sorted by actual salary · blue band = 95% CI · dashed = posterior mean",
    x        = "Player rank (by actual salary)",
    y        = "Salary (USD millions, log scale)"
  ) +

  theme_bw(base_size = 13) +
  theme(
    legend.position   = "top",
    legend.key.size   = unit(1.2, "lines"),
    strip.background  = element_rect(fill = "#e8eef4"),
    strip.text        = element_text(face = "bold", size = 12),
    panel.grid.minor  = element_blank(),
    panel.grid.major  = element_line(colour = "grey90"),
    plot.title        = element_text(face = "bold", size = 14),
    plot.subtitle     = element_text(colour = "grey40", size = 10)
  )
```



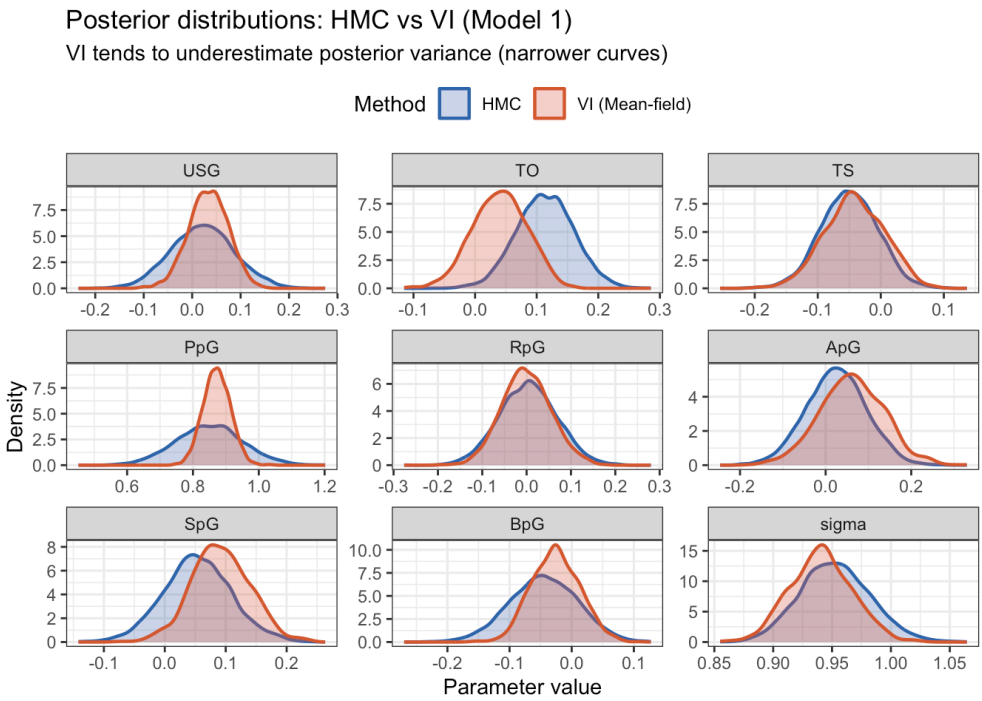
Variational Inference

```
# using model1.stan for VI
fit_m1_vi <- vb(
  stan_model("model1.stan"),
  data      = stan_data_m1,
  algorithm = "meanfield",
  seed      = 42,
  refresh = 0
)

# using model2.stan for VI
fit_m2_vi <- vb(
  stan_model("model2.stan"),
  data      = stan_data_m2,
  algorithm = "meanfield",
  seed      = 42,
  refresh = 0
)
```

Model 1 HMC and VI Comparison plot

```
library(bayesplot)
library(tidyverse)

pars_of_interest <- c(paste0("beta[", 1:8, "]"), "sigma")

draws_hmc <- as.matrix(fit_m1,    pars = pars_of_interest)
draws_vi  <- as.matrix(fit_m1_vi, pars = pars_of_interest)

beta_labels <- c("USG","TO","TS","PpG","RpG","ApG","SpG","BpG","sigma")

# HMC vs VI的posterior density
df_hmc <- as.data.frame(draws_hmc) %>%
  pivot_longer(everything(), names_to = "parameter", values_to = "value") %>%
  mutate(method = "HMC")

df_vi <- as.data.frame(draws_vi) %>%
  pivot_longer(everything(), names_to = "parameter", values_to = "value") %>%
  mutate(method = "VI (Mean-field)")

df_combined <- bind_rows(df_hmc, df_vi)
```

```r
param_map <- setNames(beta_labels, pars_of_interest)
df_combined$parameter <- param_map[df_combined$parameter]
df_combined$parameter <- factor(df_combined$parameter, levels = beta_labels)

ggplot(df_combined, aes(x = value, fill = method, color = method)) +
  geom_density(alpha = 0.3, linewidth = 0.8) +
  facet_wrap(~parameter, scales = "free", ncol = 3) +
  scale_fill_manual(values  = c("HMC" = "#3266ad", "VI (Mean-field)" = "#D85A30")) +
  scale_color_manual(values = c("HMC" = "#3266ad", "VI (Mean-field)" = "#D85A30")) +
  labs(
    title    = "Posterior distributions: HMC vs VI (Model 1)",
    subtitle = "VI tends to underestimate posterior variance (narrower curves)",
    x = "Parameter value", y = "Density", fill = "Method", color = "Method"
  ) +
  theme_bw() +
  theme(legend.position = "top")
```

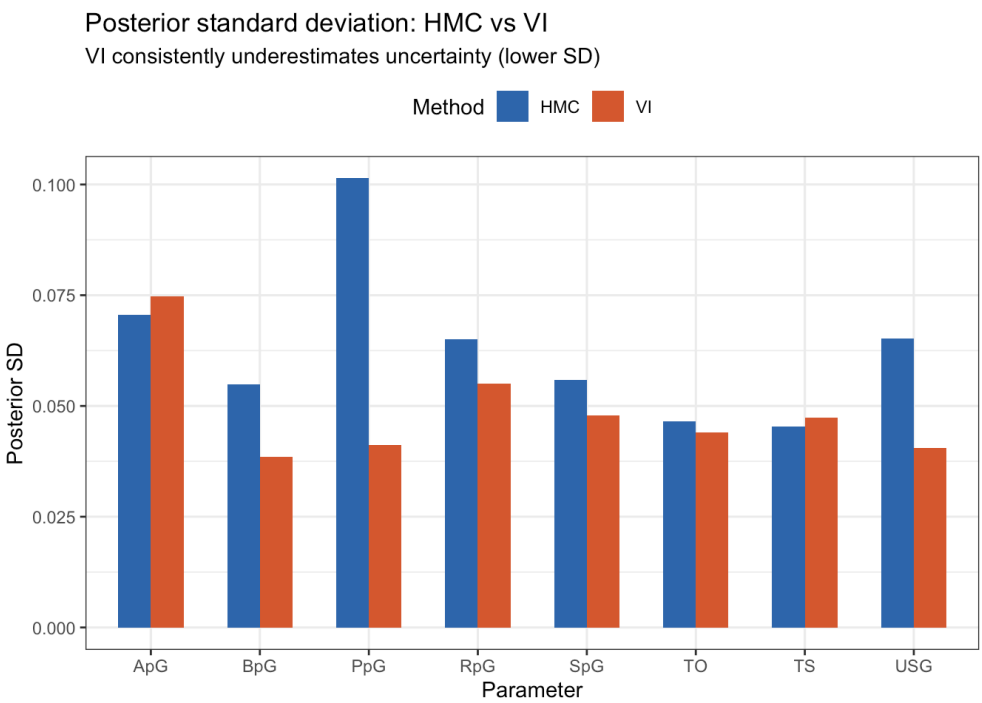



```r
# quantify the mean of posterior and difference between SD
comparison_df <- data.frame(
  Parameter = beta_labels,
  HMC_mean  = colMeans(draws_hmc),
  VI_mean   = colMeans(draws_vi),
  HMC_sd    = apply(draws_hmc, 2, sd),
  VI_sd     = apply(draws_vi,  2, sd)
) %>%
  mutate(
    mean_diff  = round(abs(HMC_mean - VI_mean), 4),
    sd_ratio   = round(VI_sd / HMC_sd, 3),       # <1 represents VI undermines variance
    VI_narrower = ifelse(sd_ratio < 1, "Yes", "No")
  )

print(comparison_df)
```

```
        Parameter     HMC_mean      VI_mean     HMC_sd      VI_sd mean_diff
beta[1]       USG  0.021590187  0.033360171 0.06515496 0.04050515    0.0118
beta[2]        TO  0.113848006  0.040083965 0.04653124 0.04399704    0.0738
beta[3]        TS -0.049966986 -0.041891458 0.04537129 0.04735256    0.0081
beta[4]       PpG  0.849932138  0.872499966 0.10137485 0.04122671    0.0226
beta[5]       RpG  0.003263227 -0.000494899 0.06508981 0.05501257    0.0038
beta[6]       ApG  0.024909304  0.061085795 0.07064188 0.07475477    0.0362
beta[7]       SpG  0.053424223  0.092205880 0.05590009 0.04783674    0.0388
beta[8]       BpG -0.045066400 -0.025860580 0.05481675 0.03850121    0.0192
sigma       sigma  0.952633693  0.940695047 0.02994793 0.02635950    0.0119
```

```
        sd_ratio VI_narrower
beta[1]    0.622         Yes
beta[2]    0.946         Yes
beta[3]    1.044          No
beta[4]    0.407         Yes
beta[5]    0.845         Yes
beta[6]    1.058          No
beta[7]    0.856         Yes
beta[8]    0.702         Yes
sigma      0.880         Yes
```

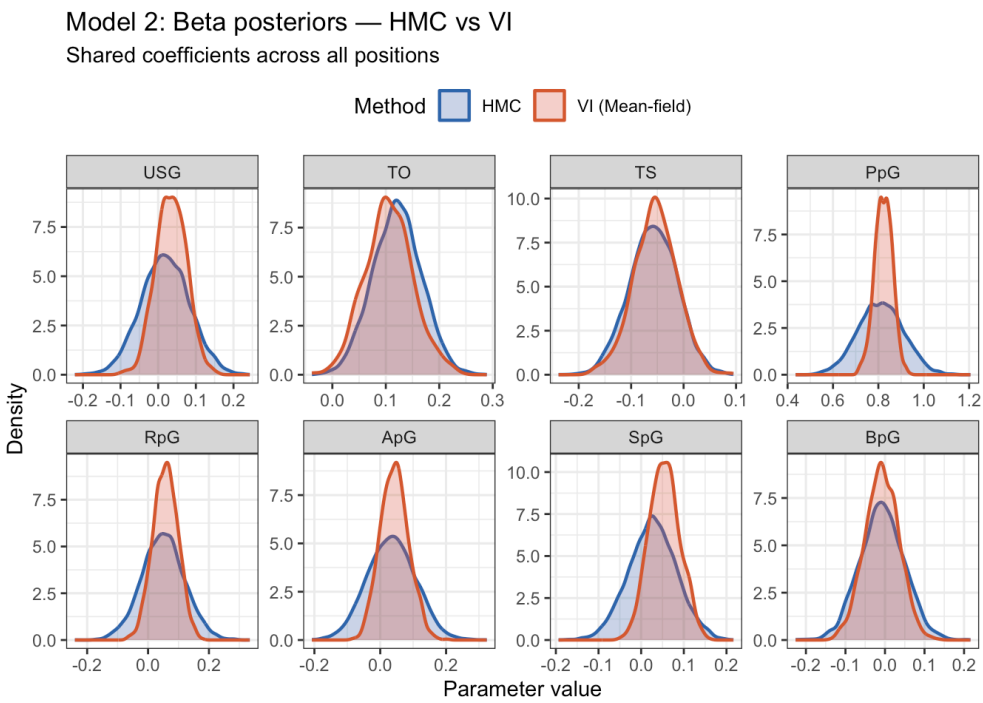
```r
# visualize SD comparison
comparison_df %>%
  filter(Parameter != "sigma") %>%
  select(Parameter, HMC_sd, VI_sd) %>%
  pivot_longer(-Parameter, names_to = "Method", values_to = "SD") %>%
  mutate(Method = ifelse(Method == "HMC_sd", "HMC", "VI")) %>%
  ggplot(aes(x = Parameter, y = SD, fill = Method)) +
  geom_bar(stat = "identity", position = "dodge", width = 0.6) +
  scale_fill_manual(values = c("HMC" = "#3266ad", "VI" = "#D85A30")) +
  labs(
    title    = "Posterior standard deviation: HMC vs VI",
    subtitle = "VI consistently underestimates uncertainty (lower SD)",
    x = "Parameter", y = "Posterior SD"
  ) +
  theme_bw() +
  theme(legend.position = "top")
```



Model 2 HMC and VI Comparison plot

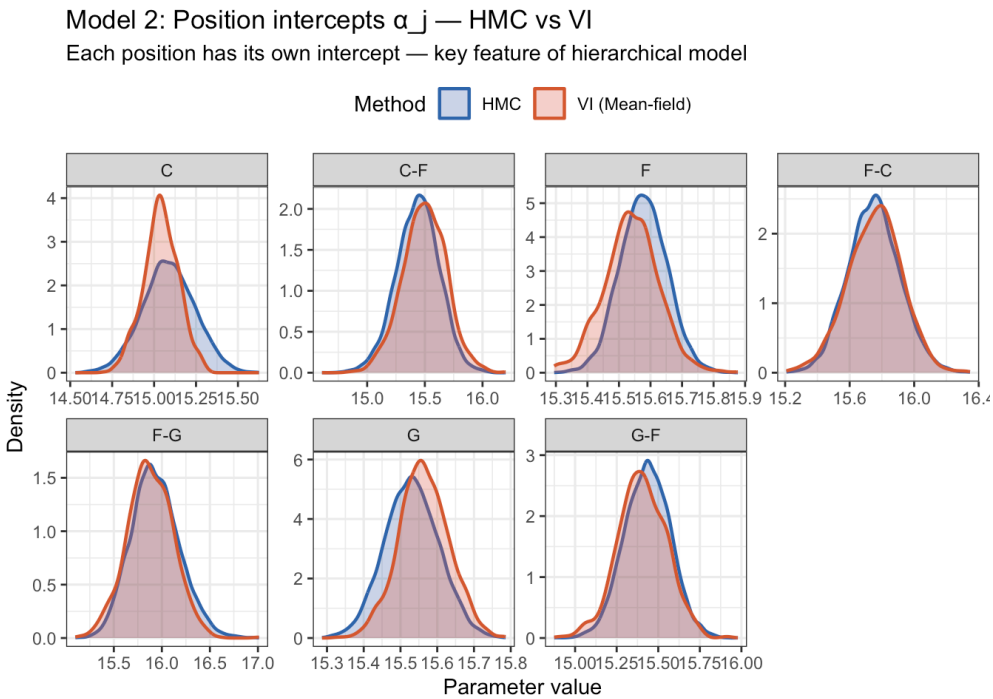
```r
# extract posterior samples
pars_beta  <- paste0("beta[",  1:8, "]")
pars_alpha <- paste0("alpha[", 1:7, "]")
pars_hyper <- c("mu_a", "tau", "sigma")

pos_labels  <- pos_levels # 7 positions in total
beta_labels <- c("USG","TO","TS","PpG","RpG","ApG","SpG","BpG")

# comparison for Beta coefficient (HMC vs VI)
draws_hmc_beta <- as.matrix(fit_m2,    pars = pars_beta)
draws_vi_beta  <- as.matrix(fit_m2_vi, pars = pars_beta)

df_hmc_beta <- as.data.frame(draws_hmc_beta) %>%
  pivot_longer(everything(), names_to = "parameter", values_to = "value") %>%
```

```r
  mutate(method = "HMC")

df_vi_beta <- as.data.frame(draws_vi_beta) %>%
  pivot_longer(everything(), names_to = "parameter", values_to = "value") %>%
  mutate(method = "VI (Mean-field)")

param_map_beta <- setNames(beta_labels, pars_beta)

bind_rows(df_hmc_beta, df_vi_beta) %>%
  mutate(parameter = param_map_beta[parameter],
         parameter = factor(parameter, levels = beta_labels)) %>%
  ggplot(aes(x = value, fill = method, color = method)) +
  geom_density(alpha = 0.3, linewidth = 0.8) +
  facet_wrap(~parameter, scales = "free", ncol = 4) +
  scale_fill_manual(values  = c("HMC" = "#3266ad", "VI (Mean-field)" = "#D85A30")) +
  scale_color_manual(values = c("HMC" = "#3266ad", "VI (Mean-field)" = "#D85A30")) +
  labs(
    title    = "Model 2: Beta posteriors — HMC vs VI",
    subtitle = "Shared coefficients across all positions",
    x = "Parameter value", y = "Density",
    fill = "Method", color = "Method"
  ) +
  theme_bw() +
  theme(legend.position = "top")
```

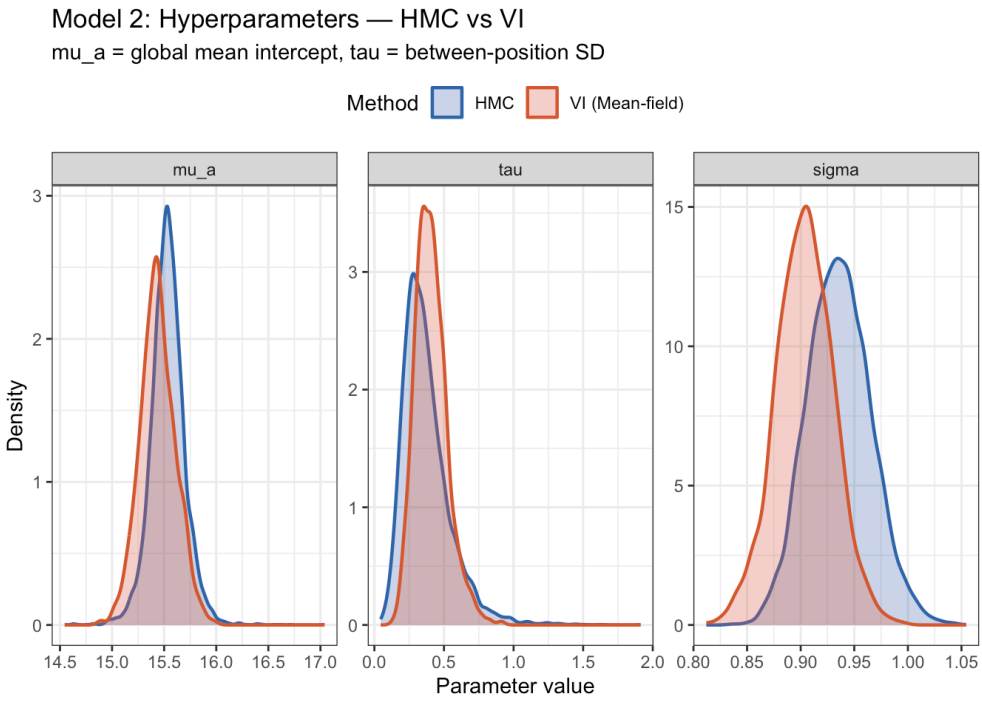



```r
# compare: Position intercepts alpha[j]  ————————————
draws_hmc_alpha <- as.matrix(fit_m2,    pars = pars_alpha)
draws_vi_alpha  <- as.matrix(fit_m2_vi, pars = pars_alpha)

param_map_alpha <- setNames(pos_labels, pars_alpha)

df_hmc_alpha <- as.data.frame(draws_hmc_alpha) %>%
  pivot_longer(everything(), names_to = "parameter", values_to = "value") %>%
  mutate(method = "HMC")

df_vi_alpha <- as.data.frame(draws_vi_alpha) %>%
  pivot_longer(everything(), names_to = "parameter", values_to = "value") %>%
  mutate(method = "VI (Mean-field)")

bind_rows(df_hmc_alpha, df_vi_alpha) %>%
  mutate(parameter = param_map_alpha[parameter],
         parameter = factor(parameter, levels = pos_labels)) %>%
  ggplot(aes(x = value, fill = method, color = method)) +
  geom_density(alpha = 0.3, linewidth = 0.8) +
  facet_wrap(~parameter, scales = "free", ncol = 4) +
```

```
  scale_fill_manual(values  = c("HMC" = "#3266ad", "VI (Mean-field)" = "#D85A30")) +
  scale_color_manual(values = c("HMC" = "#3266ad", "VI (Mean-field)" = "#D85A30")) +
  labs(
    title    = "Model 2: Position intercepts α_j — HMC vs VI",
    subtitle = "Each position has its own intercept — key feature of hierarchical model",
    x = "Parameter value", y = "Density",
    fill = "Method", color = "Method"
  ) +
  theme_bw() +
  theme(legend.position = "top")
```




```
# compare Hyperparameters: mu_a, tau, sigma
draws_hmc_hyper <- as.matrix(fit_m2,    pars = pars_hyper)
draws_vi_hyper  <- as.matrix(fit_m2_vi, pars = pars_hyper)

df_hmc_hyper <- as.data.frame(draws_hmc_hyper) %>%
  pivot_longer(everything(), names_to = "parameter", values_to = "value") %>%
  mutate(method = "HMC")

df_vi_hyper <- as.data.frame(draws_vi_hyper) %>%
  pivot_longer(everything(), names_to = "parameter", values_to = "value") %>%
  mutate(method = "VI (Mean-field)")

bind_rows(df_hmc_hyper, df_vi_hyper) %>%
  mutate(parameter = factor(parameter, levels = pars_hyper)) %>%
  ggplot(aes(x = value, fill = method, color = method)) +
  geom_density(alpha = 0.3, linewidth = 0.8) +
  facet_wrap(~parameter, scales = "free", ncol = 3) +
  scale_fill_manual(values  = c("HMC" = "#3266ad", "VI (Mean-field)" = "#D85A30")) +
  scale_color_manual(values = c("HMC" = "#3266ad", "VI (Mean-field)" = "#D85A30")) +
  labs(
    title    = "Model 2: Hyperparameters — HMC vs VI",
    subtitle = "mu_a = global mean intercept, tau = between-position SD",
    x = "Parameter value", y = "Density",
    fill = "Method", color = "Method"
  ) +
  theme_bw() +
  theme(legend.position = "top")
```

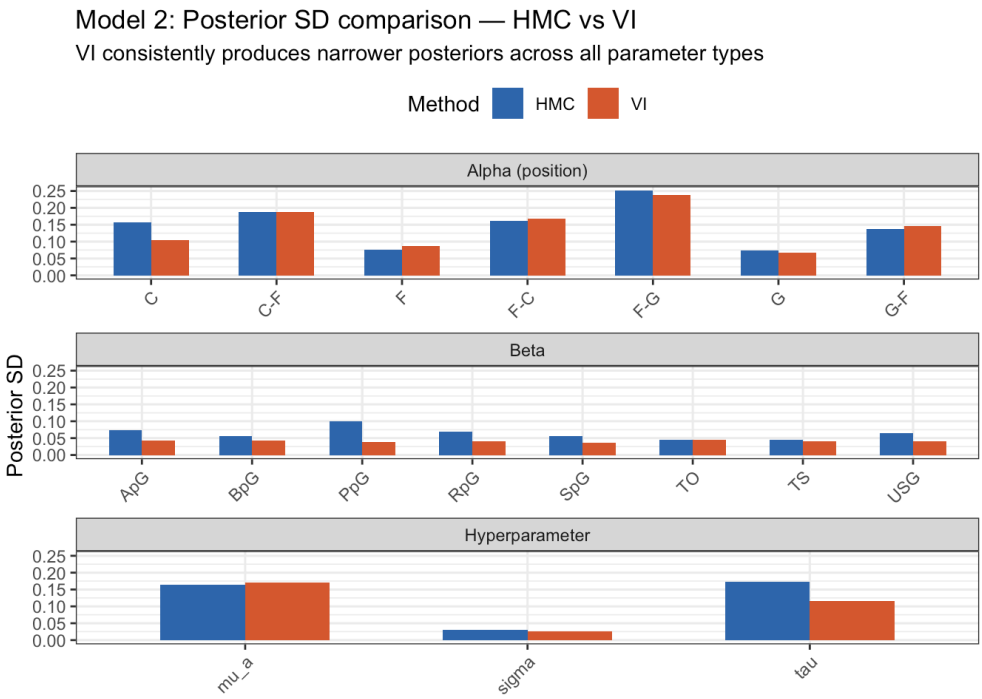



```r
# Quantify the SD comparison for all parameters
all_pars   <- c(pars_beta, pars_alpha, pars_hyper)
all_labels <- c(beta_labels, pos_labels, pars_hyper)

draws_hmc_all <- as.matrix(fit_m2,    pars = all_pars)
draws_vi_all  <- as.matrix(fit_m2_vi, pars = all_pars)

comparison_m2 <- data.frame(
  Parameter = all_labels,
  Type      = c(rep("Beta", 8), rep("Alpha (position)", 7), rep("Hyperparameter", 3)),
  HMC_mean  = colMeans(draws_hmc_all),
  VI_mean   = colMeans(draws_vi_all),
  HMC_sd    = apply(draws_hmc_all, 2, sd),
  VI_sd     = apply(draws_vi_all,  2, sd)
) %>%
  mutate(
    sd_ratio    = round(VI_sd / HMC_sd, 3),
    VI_narrower = ifelse(sd_ratio < 1, "Yes", "No"),
    mean_diff   = round(abs(HMC_mean - VI_mean), 4)
  )

print(comparison_m2)
```

```
         Parameter              Type     HMC_mean     VI_mean     HMC_sd
beta[1]        USG              Beta  0.018680506  0.03424573 0.06457481
beta[2]         TO              Beta  0.120971522  0.10585604 0.04555390
beta[3]         TS              Beta -0.057812232 -0.05389942 0.04570086
beta[4]        PpG              Beta  0.810567744  0.82251054 0.09996224
beta[5]        RpG              Beta  0.049358704  0.05698512 0.07010369
beta[6]        ApG              Beta  0.034067102  0.04347502 0.07307278
beta[7]        SpG              Beta  0.026953826  0.05454884 0.05624376
beta[8]        BpG              Beta -0.007355687 -0.00781191 0.05632581
alpha[1]         C Alpha (position) 15.087109514 15.04119570 0.15656417
alpha[2]       C-F Alpha (position) 15.438669914 15.49566340 0.18678547
alpha[3]         F Alpha (position) 15.577328098 15.53906320 0.07504110
alpha[4]       F-C Alpha (position) 15.751444951 15.75536020 0.16079338
alpha[5]       F-G Alpha (position) 15.932648279 15.87814340 0.25131605
alpha[6]         G Alpha (position) 15.530949692 15.56363010 0.07384719
alpha[7]       G-F Alpha (position) 15.434335087 15.40407180 0.13733224
mu_a          mu_a   Hyperparameter 15.533412802 15.43854140 0.16459601
tau            tau   Hyperparameter  0.374015993  0.40784824 0.17369696
sigma        sigma   Hyperparameter  0.935985885  0.90367339 0.02951766
              VI_sd sd_ratio VI_narrower mean_diff
beta[1]  0.04062844    0.629         Yes    0.0156
beta[2]  0.04488684    0.985         Yes    0.0151
```

```
beta[3]  0.04161763    0.911         Yes    0.0039
beta[4]  0.03852929    0.385         Yes    0.0119
beta[5]  0.04050299    0.578         Yes    0.0076
beta[6]  0.04268735    0.584         Yes    0.0094
beta[7]  0.03559845    0.633         Yes    0.0276
beta[8]  0.04272420    0.759         Yes    0.0005
alpha[1] 0.10411867    0.665         Yes    0.0459
alpha[2] 0.18864938    1.010          No    0.0570
alpha[3] 0.08656492    1.154          No    0.0383
alpha[4] 0.16779864    1.044          No    0.0039
alpha[5] 0.23742863    0.945         Yes    0.0545
alpha[6] 0.06770697    0.917         Yes    0.0327
alpha[7] 0.14540767    1.059          No    0.0303
mu_a     0.17001578    1.033          No    0.0949
tau      0.11482948    0.661         Yes    0.0338
sigma    0.02674266    0.906         Yes    0.0323
```

```r
# SD comparison visualization by bar chart(differs in position)
comparison_m2 %>%
  select(Parameter, Type, HMC_sd, VI_sd) %>%
  pivot_longer(c(HMC_sd, VI_sd), names_to = "Method", values_to = "SD") %>%
  mutate(Method = ifelse(Method == "HMC_sd", "HMC", "VI")) %>%
  ggplot(aes(x = Parameter, y = SD, fill = Method)) +
  geom_bar(stat = "identity", position = "dodge", width = 0.6) +
  facet_wrap(~Type, scales = "free_x", ncol = 1) +
  scale_fill_manual(values = c("HMC" = "#3266ad", "VI" = "#D85A30")) +
  labs(
    title    = "Model 2: Posterior SD comparison — HMC vs VI",
    subtitle = "VI consistently produces narrower posteriors across all parameter types",
    x = NULL, y = "Posterior SD"
  ) +
  theme_bw() +
  theme(
    legend.position = "top",
    axis.text.x     = element_text(angle = 45, hjust = 1)
  )
```



Stan Code for Model 1

```
data {
  int<lower=0> N; // number of observations
  int<lower=0> K; // number of betas
  matrix[N, K] X; // predictor matrix (standardized)
  vector[N] y;    // log(Salary)
```

```
}

parameters {
  real alpha;
  vector[K] beta;
  real<lower=0> sigma;
}

model {
  // Priors
  alpha ~ normal(16, 2);
  beta  ~ normal(0, 1);
  sigma ~ normal(0, 1);

  // Likelihood
  y ~ normal(X * beta + alpha, sigma);
}

generated quantities {
  // Posterior predictive distribution
  vector[N] y_rep;    // posterior predictive distribution data - result
  vector[N] log_lik;  // pointwise log-likelihood
  vector[N] residual; // residuals on log scale

  for (n in 1:N) {
    real mu_n = alpha + X[n] * beta;
    y_rep[n]   = normal_rng(mu_n, sigma);
    log_lik[n] = normal_lpdf(y[n] | mu_n, sigma);
    residual[n] = y[n] - mu_n;
  }
}
```

Stan Code for Model 2

```
data {
  int<lower=0> N; // number of observations
  int<lower=0> K; // number of betas
  int<lower=0> J; // number of positions
  matrix[N, K] X; // predictor matrix (standardized)
  vector[N] y;    // log(Salary)
  array[N] int<lower=1, upper=J> pos_id;  // position index for each player
}

parameters {
  // Hyperparameters (group-level)
  real mu_a; // mean of position intercepts
  real<lower=0> tau; // SD of position intercepts

  // Group-level parameters
  vector[J] alpha; // position-specific intercepts

  // Population-level parameters
  vector[K] beta; // shared regression coefficients
  real<lower=0> sigma; // residual SD
}

model {
  // Hyperpriors
  mu_a ~ normal(16, 2);
  tau  ~ normal(0, 1);

  // Group-level priors (partial pooling)
  alpha ~ normal(mu_a, tau);

  // Priors on coefficients
  beta  ~ normal(0, 1);
  sigma ~ exponential(1);

  // Likelihood
  for (n in 1:N) {
    y[n] ~ normal(alpha[pos_id[n]] + X[n] * beta, sigma);
```

```
  }
}

generated quantities {
  // Posterior predictive distribution
  vector[N] y_rep;    // posterior predictive distribution data - result
  vector[N] log_lik;  // pointwise log-likelihood
  vector[N] residual; // residuals on log scale

  for (n in 1:N) {
    real mu_n = alpha[pos_id[n]] + X[n] * beta;
    y_rep[n]   = normal_rng(mu_n, sigma);
    log_lik[n] = normal_lpdf(y[n] | mu_n, sigma);
    residual[n] = y[n] - mu_n;
  }
}
```